Projektarbeit

Fachbereich MNI – Informatik | WS 2025/2026

Analyse und Evaluation des Follow-the-Gap-Algorithmus zur autonomen Hindernisvermeidung auf einem Jetson-Nano-basierten MXCarkit

Autoren:	Aly Elbermawy (5335848)
	Omar Elalami (5379642)
	Zakaria El-aboudi (5223183)
Betreuer:	Prof. Dr. Benedikt Lattke
Hochschule:	Technische Hochschule Mittelhessen, Friedberg
Abgabedatum:	April 6, 2026

Kurzfassung

Diese Projektarbeit dokumentiert die Entwicklung, Integration und experimentelle Evaluation eines reaktiven, LiDAR-basierten Hindernisvermeidungssystems für das MXCarkit – ein miniaturisiertes autonomes Fahrzeug auf Basis des NVIDIA Jetson Nano. Das Ziel des Systems war es, das Fahrzeug in die Lage zu versetzen, Hindernisse selbstständig zu erkennen, ihnen auszuweichen und bei unausweichlichen Hindernissen zum Stillstand zu kommen.

Als Planungsalgorithmus wurde eine modifizierte CTU-Variante des *Follow-the-Gap*-Algorithmus (FTG) eingesetzt, der aus den Rohdaten eines RPLIDAR A1/A2-Sensors eine sichere Fahrtrichtung berechnet. Der entwickelte ROS 2-Stack umfasst sieben spezialisierte Pakete für Wahrnehmung, Planung, Adaption und Steuerung.

Die Versuche zeigten, dass das Fahrzeug Hindernisse prinzipiell erkennt und Lenkbefehle erzeugt, die korrekte Ausweichrichtungen kodieren. Im praktischen Einsatz konnte jedoch kein vollständig kollisionsfreies Fahren erreicht werden. Als Hauptursachen wurden eine fehlende TF-Koordinatentransformation zwischen Fahrzeug- und Sensorrahmen sowie eine fehlerhafte Frontbereich-Kalibrierung des LiDAR-Sensors identifiziert. Die Arbeit beschreibt den vollständigen Entwicklungs- und Debugging-Prozess und leitet konkrete Handlungsempfehlungen ab.

Schlüsselwörter: Follow-the-Gap, ROS 2, LiDAR, autonomes Fahren, Jetson Nano, MXCarkit, Hinderniserkennung, Ackermann-Steuerung

Contents

Kurzfassung **II**

Contents **III**

List of Figures **VI**

List of Tables **VII**

1 Grundlagen **1**

1.1 Hardware: Das MXCarkit 1

1.1.1 NVIDIA Jetson Nano 2

1.1.2 LiDAR-Sensor: RPLIDAR A1/A2 2

1.1.3 Ackermann-Steuermodell 3

1.2 Software-Architektur: MXCK2-Stack 3

1.2.1 ROS 2 und Docker-Container 3

1.2.2 Basisstack: mxck2_ws 4

1.3 Der Follow-the-Gap-Algorithmus 5

1.3.1 Grundprinzip 5

1.3.2 Klassisches F1TENTH-FTG vs. CTU-FTG 5

1.3.3 Algorithmus-Schritte im Detail 6

1.3.4 Hindernisrepräsentation: obstacle_substitution 7

2 Systembeschreibung und Durchführung **8**

2.1 Architektur des entwickelten FTG-Stacks 8

2.1.1 Überblick der Pakete 8

2.1.2 Topic-Schnittstellen 10

2.1.3 Konfigurationsparameter 11

2.2 Identifizierte Ausgangsprobleme und deren Behebung 11
2.2.1 Problem P1: Potenziell doppelter Planer-Output 11
2.2.2 Problem P2: Gestapelte Geschwindigkeitspolitik 12
2.3 Integration und Kompilierung 12
2.4 Iterativer Kalibrierungsprozess 13
2.4.1 Iteration 0: Initialer Start – keine Lücken gefunden 13
2.4.2 Iteration 1: front_center_deg = 90° 13
2.4.3 Iteration 2: Verengung des Sichtfelds 14
2.4.4 Iteration 3: Vollständige Kalibrierung (front_center_deg = 0°) 15
2.5 Praktischer Fahrttest 16
2.5.1 Manueller Fahrttest 16
2.5.2 Autonomer Fahrttest 16
2.5.3 Aufgezeichnetes Rosbag 17

3 Diskussion und Fehleranalyse 18
3.1 Übersicht der identifizierten Fehlerursachen 18
3.2 F1: Fehlende TF-Transformation 18
3.2.1 Beschreibung 18
3.2.2 Ursache 19
3.2.3 Beleg aus Messdaten 19
3.3 F2: Falsch kalibriertes Frontfenster 20
3.3.1 Beschreibung 20
3.3.2 Auswirkung auf den Lenkwinkel 20
3.3.3 Verbleibende Einschränkung 21
3.4 F3: Zu kleiner Hindernissicherheitsradius 21
3.4.1 Beschreibung 21
3.4.2 Konsequenz 21
3.5 F4: Fehlende Hindernisgruppierung 21
3.5.1 Beschreibung 21
3.5.2 Konsequenz 22
3.6 F5: ROS 2-Versionskonflikt zwischen Containern 22
3.6.1 Beschreibung 22
3.6.2 Konsequenz 23

3.7 Gesamtbewertung 23
3.7.1 Handlungsempfehlungen 23

4 Fazit und Ausblick 25
4.1 Fazit 25
4.2 Ausblick 26

Bibliography 27

Literaturverzeichnis 27

Eidesstattliche Erklärung 28

List of Figures

1.1 Das MXCarkit – Versuchsfahrzeug mit RPLIDAR A1/A2 und Jetson Nano 2
1.2 ROS 2-Knoten- und Topic-Graph des vollständigen FTG-Stacks 4

2.1 Verarbeitungspipeline des FTG-Stacks (Blockdiagramm) 8
2.2 Beobachtetes Fehlverhalten: Fahrzeug reagiert in entgegengesetzte Richtung 15
2.3 Ablauf des autonomen Fahrttests 17

3.1 Einfluss des front_center_deg-Parameters auf den berechneten Zielwinkel 20
3.2 Hindernisrepräsentation ohne Clustering – jeder LiDAR-Messpunkt ist ein separates Objekt 22

List of Tables

1.1 Technische Spezifikationen des MXCarkit 1
1.2 Technische Kenndaten des RPLIDAR A1/A2 3
1.3 Docker-Container-Übersicht auf dem Jetson Nano 4
1.4 Vergleich: Klassisches F1TENTH-FTG vs. CTU-FTG 6

2.1 ROS 2-Pakete des FTG-Stacks und ihre Rollen 9
2.2 Kritische ROS 2-Topics in der Verarbeitungspipeline 10
2.3 Wichtige Konfigurationsparameter des Stacks 11
2.4 Messwerte bei front_center_deg = 90° (Iteration 1) 14
2.5 Messwerte nach vollständiger Kalibrierung (Iteration 3) 15

3.1 Fehlerursachen geordnet nach Schweregrad 18
3.2 Auswirkung von front_center_deg auf /final_heading_angle 20
3.3 Gesamtstatus der Systemkomponenten nach Experimenten 23

1 Grundlagen

In diesem Kapitel werden die für das Verständnis der Arbeit notwendigen Grundlagen beschrieben. Zunächst wird die verwendete Hardware vorgestellt, anschließend die zugrunde liegende Software-Architektur des MXCK-Systems und zuletzt der eingesetzte Follow-the-Gap-Algorithmus erläutert.

1.1 Hardware: Das MXCarkit

Das MXCarkit ist ein miniaturisiertes autonomes Versuchsfahrzeug, das im Rahmen von Lehrveranstaltungen an der Technischen Hochschule Mittelhessen eingesetzt wird. Es dient als Plattform für die Erprobung autonomer Fahrfunktionen in realen Umgebungen.

Table 1.1: Technische Spezifikationen des MXCarkit

Eigenschaft	**Wert**
Fahrzeugbreite	$> 30\,\text{cm}$
Fahrzeuglänge	$> 50\,\text{cm}$
Antrieb	Bürstenloser Elektromotor
Motorcontroller	VESC (Vedder Electronic Speed Controller)
Lenkung	Ackermann-Geometrie
Recheneinheit	NVIDIA Jetson Nano
LiDAR-Sensor	RPLIDAR A1/A2
Betriebssystem	Ubuntu 18.04 (L4T)
Middleware	ROS 2 (Foxy / Humble)

[FOTO EINFÜGEN]

Foto des MXCarkit von vorne und seitlich mit beschrifteten Komponenten (LiDAR, Jetson Nano, VESC, Räder)

Figure 1.1: Das MXCarkit – Versuchsfahrzeug mit RPLIDAR A1/A2 und Jetson Nano

1.1.1 NVIDIA Jetson Nano

Der Jetson Nano ist ein eingebetteter Einplatinencomputer der Firma NVIDIA mit einer quad-core ARM Cortex-A57 CPU und einer integrierten 128-core Maxwell-GPU. Im Kontext dieser Arbeit übernimmt er die vollständige Verarbeitungskette von der Sensordatenaufnahme über die ROS 2-Middleware bis hin zur Ausgabe von Steuerkommandos an den VESC.

1.1.2 LiDAR-Sensor: RPLIDAR A1/A2

Der RPLIDAR A1/A2 ist ein kostengünstiger 2D-LiDAR-Sensor, der per Triangulationsverfahren Distanzmessungen in einem vollständigen 360-Sichtfeld liefert. Relevante Parameter für diese Arbeit:

Table 1.2: Technische Kenndaten des RPLIDAR A1/A2

Parameter	**Wert**
Scanrate	$\approx 11\,\mathrm{Hz}$
Messpunkte/Umdrehung	≈ 1800
Winkelauflösung	$\approx 0{,}35$
Minimalmessreichweite	$0{,}15\,\mathrm{m}$
Maximalmessreichweite	12–$16\,\mathrm{m}$
Sichtfeld	360
Schnittstelle	USB (seriell)

Der Sensor ist auf dem Fahrzeugdach montiert und publiziert Messdaten über den ROS 2-Topic `/scan` im Format `sensor_msgs/msg/LaserScan`. Der Scan beginnt bei `angle_min` $= -\pi$ und endet bei `angle_max` $= +\pi$.

Wichtige Konvention: Im Koordinatensystem des RPLIDAR entspricht der Winkel 0 *nicht* automatisch der Fahrzeugvorderseite. Die tatsächliche Zuordnung hängt von der physischen Montageposition des Sensors ab und muss kalibriert werden (siehe Abschnitt 2.4).

1.1.3 Ackermann-Steuermodell

Das Fahrzeug verwendet die Ackermann-Lenkgeometrie, bei der Vorder- und Hinterräder unterschiedliche Lenkwinkel aufweisen, um schlupffreies Kurvenfahren zu ermöglichen. Steuerkommandos werden im ROS 2-Format `ackermann_msgs/msg/AckermannDriveStam` mit den Feldern `steering_angle` (Lenkwinkel in Radiant) und `speed` (Geschwindigkeit in m/s) übergeben.

1.2 Software-Architektur: MXCK2-Stack

1.2.1 ROS 2 und Docker-Container

Das MXCarkit verwendet ROS 2 (Robot Operating System 2) als Middleware. Die gesamte Softwareumgebung ist in Docker-Containern gekapselt, was eine reproduzierbare Deploymentumgebung gewährleistet. Für diese Arbeit relevante Container sind:

Table 1.3: Docker-Container-Übersicht auf dem Jetson Nano

Container	**ROS 2-Version**	**Inhalt**
`mxck2_control`	Humble	Fahrzeugsteuerung, VESC-Treiber, LiDAR-Treiber
`mxck2_development`	Foxy	Entwicklungsumgebung, FTG-Stack (diese Arbeit)
`mxck2_lidar`	Humble	Alternativer LiDAR-Treiber

Der Workspace `/mxck2_ws` im Entwicklungscontainer ist über ein Docker-Volume direkt aus dem Host-Verzeichnis `/home/mxck/development_ws` eingehängt, sodass Codeänderungen ohne Neustart des Containers wirksam werden.

1.2.2 Basisstack: mxck2_ws

Der vorhandene Basisstack (Repository: `william-mx/mxck2_ws`) stellt die grundlegende Fahrzeugfunktionalität bereit: manuelle RC-Steuerung, URDF-Fahrzeugbeschreibung, Kamerabilder und Basistreiber. Er definiert außerdem den TF-Baum (Transform Tree) des Fahrzeugs mit den Koordinatenrahmen `base_link` (Fahrzeugmittelpunkt) und `laser` (LiDAR-Sensor).

[FOTO EINFÜGEN]

Screenshot des ROS 2-Topic-Graphs (z.B. aus Foxglove) mit allen aktiven Knoten und Verbindungen

Figure 1.2: ROS 2-Knoten- und Topic-Graph des vollständigen FTG-Stacks

1.3 Der Follow-the-Gap-Algorithmus

1.3.1 Grundprinzip

Der Follow-the-Gap-Algorithmus (FTG) ist ein reaktives Planungsverfahren für autonome Fahrsysteme, das ohne globale Karte oder Pfadplanung auskommt. Er operiert ausschließlich auf aktuellen Sensordaten und berechnet in Echtzeit eine kollisionsfreie Fahrtrichtung.

Das Grundprinzip lässt sich in drei Schritten zusammenfassen:

1. Identifiziere alle Hindernisse im Sichtfeld des Sensors.
2. Finde die größte *Lücke* (freien Winkelbereich) zwischen benachbarten Hindernissen.
3. Steuere das Fahrzeug in Richtung des Mittelpunkts dieser Lücke.

$$\theta_{\text{Ziel}} = \frac{\theta_{\text{links}} + \theta_{\text{rechts}}}{2} \tag{1.1}$$

Dabei sind θ_{links} und θ_{rechts} die Winkel der linken und rechten Begrenzung der gewählten Lücke.

1.3.2 Klassisches F1TENTH-FTG vs. CTU-FTG

Im Rahmen dieser Arbeit wurde eine modifizierte Variante des FTG-Algorithmus verwendet, die ursprünglich von der Tschechischen Technischen Universität Prag (CTU) entwickelt wurde. Im Unterschied zur klassischen F1TENTH-Implementierung arbeitet diese Variante nicht direkt auf Laserscan-Rohdaten, sondern auf einer vorverarbeiteten Liste kreisförmiger Hindernisse.

Table 1.4: Vergleich: Klassisches F1TENTH-FTG vs. CTU-FTG

Merkmal	**Klassisches F1TENTH-FTG**	**CTU-FTG (diese Arbeit)**
Eingabe	Rohlaserscan (`LaserScan`)	Kreishindernisliste (`ObstaclesStamped`)
Sicherheitsradius	Explizit um jeden Messpunkt (≈ halbe Fahrzeugbreite)	Implizit durch Hindernisradius (0,01 m)
Lückenbewertung	Tiefster freier Strahl	Größte Winkelbreite
Implementierung	Python oder C++	C++
Geschwindigkeit	Oft direkt integriert	Separater Adapterknoten

1.3.3 Algorithmus-Schritte im Detail

Der CTU-FTG-Kern (`follow_the_gap_v0`) durchläuft pro Planungszyklus folgende Schritte:

Schritt 1. Eingabe konvertieren: Kreisförmige Hindernisse aus `ObstaclesStamped` in interne `Obstacle`-Objekte umwandeln.

Schritt 2. Filtern: Hindernisse außerhalb des Sichtfelds oder jenseits der maximalen Relevanzreichweite verwerfen.

Schritt 3. Lücken konstruieren: Zwischen je zwei benachbarten, nicht-überlappenden Hindernissen ein `Gap`-Objekt anlegen.

Schritt 4. Beste Lücke wählen: Die Lücke mit der größten Winkelbreite auswählen, sofern sie breit genug für das Fahrzeug ist.

Schritt 5. Zielwinkel berechnen: Mittelpunktwinkel der Lücke nach Gleichung (1.1) berechnen.

Schritt 6. Fallback: Falls keine Lücke gefunden wird, Corner-Following aktivieren und `/gap_found = false` publizieren.

1.3.4 Hindernisrepräsentation: obstacle_substitution

Ein wesentlicher Bestandteil des Systems ist der Knoten `obstacle_substitution_node`, der jeden einzelnen LiDAR-Messstahl in ein kreisförmiges Hindernis mit Radius $r = 0{,}01\,\mathrm{m}$ umwandelt. Die kartesischen Koordinaten werden aus den Polarkoordinaten des Scans berechnet:

$$x = d \cdot \cos(\theta), \quad y = d \cdot \sin(\theta) \tag{1.2}$$

wobei d die gemessene Distanz und θ der zugehörige Scanwinkel ist.

2 Systembeschreibung und Durchführung

Dieses Kapitel beschreibt den entwickelten FTG-Stack im Detail, den Integrationsprozess und die iterativen Schritte zur Fehlerdiagnose und Kalibrierung. Wo Messdaten oder Fotos noch ausstehen, sind entsprechende Platzhalter markiert.

2.1 Architektur des entwickelten FTG-Stacks

2.1.1 Überblick der Pakete

Der entwickelte Stack (`ftg_mxck`) ergänzt den Basisstack des MXCarkit um sieben spezialisierte ROS 2-Pakete. Der vollständige Datenpfad ist in Abbildung 2.1.1 schematisch dargestellt.

[FOTO EINFÜGEN]

Blockdiagramm: /scan → scan_preprocessor → obstacle_substitution → follow_the_gap_v0 → ctu_ftg_adapter → ftg_command_node → /autonomous/ackermann_cmd

Figure 2.1: Verarbeitungspipeline des FTG-Stacks (Blockdiagramm)

Table 2.1: ROS 2-Pakete des FTG-Stacks und ihre Rollen

Paket	**Aufgabe**
`obstacle_msgs`	Benutzerdefinierte Nachrichtentypen für Hindernisdarstellung
`obstacle_substitution`	Umwandlung jedes LiDAR-Strahls in ein Kreishindernis
`follow_the_gap_v0`	CTU-FTG-Planerkern (C++): berechnet sicheren Durchfahrtswinkel
`mxck_ftg_perception`	Optionale LiDAR-Vorverarbeitung auf Frontbereich
`mxck_ftg_planner`	Adapter: Winkel begrenzen, Geschwindigkeit nach Abstand wählen
`mxck_ftg_control`	Steuerknoten: Ackermann-Kommandos aus Winkel + Geschwindigkeit
`mxck_ftg_bringup`	Launch-Dateien zur koordinierten Inbetriebnahme

2.1.2 Topic-Schnittstellen

Table 2.2: Kritische ROS 2-Topics in der Verarbeitungspipeline

Topic	**Typ**	**Bedeutung**
`/scan`	LaserScan	Rohdaten des RPLIDAR
`/autonomous/ftg/scan_filtered`	LaserScan	Gefilterter Frontscan
`/obstacles`	ObstaclesStamped	Kreishindernisse
`/final_heading_angle`	Float32	FTG-Zielwinkel (rad)
`/gap_found`	Bool	Planerstatus
`/autonomous/ftg/gap_angle`	Float32	Begrenzter Lenkwinkel (rad)
`/autonomous/ftg/target_speed`	Float32	Zielgeschwindigkeit (m/s)
`/autonomous/ackermann_cmd`	AckermannDriveStamped	Finales Fahrkommando

2.1.3 Konfigurationsparameter

Table 2.3: Wichtige Konfigurationsparameter des Stacks

Parameter	Endwert	Beschreibung
`front_center_deg`	0	LiDAR-Montagewinkel zur Fahrzeugfront
`front_fov_deg`	60	Breite des ausgewerteten Frontbereichs
`clip_min_range_m`	0,18 m	Minimale Messreichweite
`clip_max_range_m`	2,0 m	Maximale Messreichweite
`stop_distance_m`	0,30 m	Vollbremsung bei Unterschreitung
`slow_distance_m`	0,55 m	Langsamfahrt bei Unterschreitung
`clear_distance_m`	0,90 m	Freie Fahrt bei Überschreitung
`cruise_speed_mps`	0,25 m/s	Reisegeschwindigkeit
`steering_limit_deg`	22	Maximaler Lenkwinkel
`gap_angle_limit_rad`	0,60 rad	Maximaler Lückenwinkel
`enable_turn_speed_scaling`	false	Drehwinkelbasierte Geschwindigkeitsreduktion

2.2 Identifizierte Ausgangsprobleme und deren Behebung

Bei der Analyse des bestehenden Stacks vor der eigentlichen Integration wurden mehrere architekturelle Probleme identifiziert:

2.2.1 Problem P1: Potenziell doppelter Planer-Output

Die ursprüngliche Launch-Datei `ftg_stack.launch.py` erlaubte es, sowohl den CTU-Adapterpfad als auch einen alternativen Planerpfad (`ftg_planner_node`) gleichzeitig zu starten. Bei gleichzeitigem Betrieb beider Pfade würden mehrere Knoten auf denselben Output-Topics publizieren, was zu undefiniertem Steuerverhalten führt.

Lösung: Einführung des Launch-Parameters `use_ftg_planner` (Standard: `false`)

mit gegenseitigem Ausschluss – entweder der CTU-Adapter *oder* der alternative Planner-Knoten wird gestartet.

2.2.2 Problem P2: Gestapelte Geschwindigkeitspolitik

Die Standardkonfiguration aktivierte gleichzeitig die clearance-basierte Geschwindigkeitswahl im `ctu_ftg_adapter_node` *und* eine zusätzliche lenkwinkelbasierte Geschwindigkeitsskalierung im `ftg_command_node`. Das führte dazu, dass die Geschwindigkeit an zwei verschiedenen Stellen reduziert wurde, was die Kalibrierung deutlich erschwerte.

Lösung: Deaktivierung der redundanten Skalierung durch Setzen von `enable_turn_speed_scali` `false` in `ftg_control.yaml`.

2.3 Integration und Kompilierung

Der Stack wurde im Docker-Container `mxck2_development` (ROS 2 Foxy, Python 3.8, CMake) kompiliert. Der Kompilierungsbefehl:

Listing 2.1: Build-Befehl für den FTG-Stack

```
colcon build --symlink-install \
  --packages-select \
    obstacle_msgs \
    obstacle_substitution \
    follow_the_gap_v0 \
    mxck_ftg_perception \
    mxck_ftg_planner \
    mxck_ftg_control \
    mxck_ftg_bringup
```

Die Kompilierung dauerte auf dem Jetson Nano ca. 3 min 50 s. Das Paket `obstacle_msgs` benötigte die längste Kompilierzeit ($\approx$ 2 min 12 s) aufgrund der ROS 2-Nachrichtengenerierung. Es traten *Clock-Skew*-Warnungen auf, die auf eine Zeitdiskrepanz zwischen Host-Dateisystem und Container zurückzuführen sind, die Funktionalität jedoch nicht beeinträchtigten.

Das System wird gestartet mit:

Listing 2.2: Systemstart via Launch-Datei

```
ros2 launch mxck_ftg_bringup ftg_full_system.launch.py \
  use_vehicle_control:=false \
```

```
  record_bag:=false
```

Beim Start werden sechs ROS 2-Prozesse initiiert: `robot_state_publisher`, `scan_preprocessor` `obstacle_substitution_node`, `follow_the_gap`, `ctu_ftg_adapter_node` und `ftg_command_node`.

2.4 Iterativer Kalibrierungsprozess

Die korrekte Kalibrierung des Parameters `front_center_deg` erwies sich als zentrales Problem der Integration. Dieser Parameter gibt an, bei welchem Winkel im Rohlaserscan die Fahrzeugvorderseite liegt. Im Folgenden werden die durchgeführten Optimierungsiterationen beschrieben.

2.4.1 Iteration 0: Initialer Start – keine Lücken gefunden

Beim ersten Start des Systems lieferte der FTG-Planerkern kontinuierlich folgende Meldung:

Listing 2.3: Ausgabe des FTG-Kerns beim ersten Start

```
[follow_the_gap-4] No gaps found
[follow_the_gap-4] No gaps found
...
```

Der `/autonomous/ftg/scan_filtered`-Topic zeigte, dass der gesamte Frontbereich mit dem Minimalwert 0,05 m belegt war, also keinerlei gültige Messdaten enthielt.

Diagnose: Der Kalibrierungswert `front_center_deg = 135°` entsprach nicht der tatsächlichen Montagelage des LiDAR-Sensors. Der ausgewertete "Frontbereich" lag physikalisch *hinter* oder seitlich des Fahrzeugs. Zusätzlich fehlte die TF-Transformation `base_link` → `laser`, weshalb der Preprocessor keinen automatischen Offset berechnen konnte.

2.4.2 Iteration 1: front_center_deg = 90°

Änderung: `front_center_deg` von 135 auf 90 geändert.

Ergebnis: Der Scan-Preprocessor lieferte nun Daten, jedoch zeigte `/final_heading_angle` konstant $\approx -1{,}83$ rad – unabhängig von der Hindernisposition:

Table 2.4: Messwerte bei front_center_deg = 90° (Iteration 1)

Hindernisposition	**/final_heading (rad)**	**/gap_angle (rad)**
Vorne mittig	−1,811	−0,600 (Grenzwert)
Vorne rechts	−1,809	−0,600 (Grenzwert)
Vorne links	−1,809	−0,600 (Grenzwert)

Der Wert −0,600 entspricht exakt dem Parameter `gap_angle_limit_rad`: Der Adapter begrenzte den Winkel stets auf diesen Maximalwert, weil der Planerkern permanent −1,83 rad ausgab.

Diagnose: Der Frontbereich lag noch immer in der falschen Richtung. Die Obstacle-Koordinaten für ein Hindernis direkt vor dem Fahrzeug (ca. 35 cm Distanz) zeigten:

Listing 2.4: Obstacle-Koordinaten bei Hindernis vorne (Iteration 1)

```
center: x=0.336, y=0.862  (Hindernis "vorne mittig")
center: x=0.345, y=0.872  (Hindernis "vorne rechts")
center: x=0.328, y=0.904  (Hindernis "vorne links")
```

Alle drei Positionen liegen in der gleichen x/y-Region, das System unterscheidet nicht zwischen links, mitte und rechts. Der hohe y-Wert zeigt, dass die Fahrzeugvorderseite im Koordinatensystem entlang der y-Achse statt der x-Achse zeigt – ein klassischer Achsendrehfehler.

2.4.3 Iteration 2: Verengung des Sichtfelds

Änderungen:

- `front_fov_deg`: 100 → 60
- `clip_max_range_m`: 5,0 m → 2,0 m
- `clip_min_range_m`: auf 0,18 m (echtes LiDAR-Minimum)

Beobachtung: Das Fahrzeug zeigte erstmals Bewegungen: Es lenkte abwechselnd links und rechts und fuhr manchmal kurz vor, wenn viel freier Raum vorhanden war. Dies entsprach jedoch *nicht* dem erwarteten Ausweichverhalten.

[FOTO EINFÜGEN]

Videostandbild oder Foto aus dem Experiment: Fahrzeug lenkt in die falsche Richtung anstatt auszuweichen

Figure 2.2: Beobachtetes Fehlverhalten: Fahrzeug reagiert in entgegengesetzte Richtung

2.4.4 Iteration 3: Vollständige Kalibrierung (front_center_deg = 0°)

Änderung: `front_center_deg` von 90 auf 0 geändert.

Ergebnis: Das System lieferte nun plausible, positionsabhängige Lenkwinkel:

Table 2.5: Messwerte nach vollständiger Kalibrierung (Iteration 3)

Hindernis	**/final_heading (rad)**	**/gap_angle (rad)**	**Clearance (m)**
Karton vorne (≈ 60 cm)	+0,179	+0,182	0,906
Wand 2 m vorne	+0,179	+0,179	0,910
Hindernis vorne rechts	+0,181	+0,178	0,910
Hindernis vorne links	+0,180	+0,258	–

[DATEN / MESSUNG EINFÜGEN]

Foxglove-Screenshot oder Plot von /final_heading_angle und /gap_angle über Zeit, während ein Hindernis von links nach rechts bewegt wird – zeigt ob die Winkelreaktion korrekt ist.

Der gefilterte Scan zeigte einen sauberen, engbegrenzten Frontbereich:

Listing 2.5: Scan-Metadaten nach korrekter Kalibrierung

```
angle_min: -0.522 rad  (ca. -30 Grad)
```

```
angle_max: +0.522 rad  (ca. +30 Grad)
range_min: 0.18 m
range_max: 2.0 m
```

2.5 Praktischer Fahrttest

2.5.1 Manueller Fahrttest

Zur Verifikation der Aktuatoranbindung wurde das Fahrzeug zunächst manuell per RC-Fernbedienung gesteuert. Die Motorgeschwindigkeit (`/commands/motor/speed`) zeigte dabei Werte zwischen 700–950 RPM, was dem normalen Betriebsbereich des VESC entspricht.

Die Ausgabe auf `/autonomous/ackermann_cmd` beim manuellen Fahren:

Listing 2.6: Ackermann-Kommando während manueller Fahrt

```
drive: { steering_angle: 0.0, speed: 0.1,
         acceleration: 0.0, jerk: 0.0 }
```

2.5.2 Autonomer Fahrttest

Im autonomen Modus reagierte das Fahrzeug auf Hindernisse. Bei einem Frontobjekt in $\approx 0{,}41$ m Abstand:

Listing 2.7: Planerstatus bei nahem Frontobjekt

```
gap_found=True, heading_fresh=True, scan_fresh=True,
gap_angle_rad=+0.4788, front_clearance_m=0.408,
target_speed=0.180 m/s, reason=crawl_distance
```

Das Fahrzeug fuhr kurz geradeaus mit reduzierter Geschwindigkeit (Kriechmodus: 0,18 m/s) und lenkte leicht zur Seite. Es stieß jedoch gegen das Hindernis, bevor eine vollständige Kurskorrektur stattfinden konnte.

[FOTO EINFÜGEN]

Videostandbilder oder Bildsequenz aus dem autonomen Fahrttest: (1) Hindernis erkannt, (2) Lenkreaktion, (3) Kollision

Figure 2.3: Ablauf des autonomen Fahrttests

2.5.3 Aufgezeichnetes Rosbag

Zur späteren Analyse wurde ein 15-sekündiges Rosbag aufgezeichnet, in dem ein Hindernis vor dem Fahrzeug von links nach rechts bewegt wurde:

Listing 2.8: Rosbag-Aufnahmebefehl

```
ros2 bag record -s mcap -o /mxck2_ws/bags/ftg_diag_small \
  /scan /autonomous/ftg/scan_filtered /obstacles \
  /final_heading_angle /gap_found \
  /autonomous/ftg/gap_angle \
  /autonomous/ftg/planner_status /tf_static
```

[DATEN / MESSUNG EINFÜGEN]

Foxglove-Wiedergabe des aufgezeichneten Rosbags: Plot von /final_heading_angle und /gap_found über Zeit + 3D-Ansicht der /obstacles-Punkte. Zeigt die Reaktion des Systems auf die Hindernisbewegung.

3 Diskussion und Fehleranalyse

Dieses Kapitel analysiert systematisch, warum das System im praktischen Test keine vollständig kollisionsfreie Navigation erreichte. Die Fehlerursachen werden priorisiert und anhand der aufgezeichneten Messdaten belegt. Wo eigene Validierungsdaten noch ausstehen, sind Platzhalter für spätere Ergänzungen gesetzt.

3.1 Übersicht der identifizierten Fehlerursachen

Aus den Experimenten und der Analyse des System-Logs ergeben sich sechs identifizierbare Fehlerursachen, geordnet nach Schweregrad:

Table 3.1: Fehlerursachen geordnet nach Schweregrad

Nr.	Fehler	Schwere	Behebung
F1	Fehlende TF-Transformation (`base_link` → `laser`)	**Kritisch**	Mittel
F2	Falsch kalibriertes Frontfenster (`front_center_deg`)	**Kritisch**	Einfach
F3	Zu kleiner Hindernissicherheitsradius ($r = 0{,}01\,\mathrm{m}$)	**Hoch**	Einfach
F4	Fehlende Hindernisgruppierung (kein Clustering)	**Hoch**	Mittel
F5	ROS 2-Versionskonflikt zwischen Containern	**Hoch**	Komplex
F6	Gestapelte Geschwindigkeitspolitik	**Mittel**	Einfach

3.2 F1: Fehlende TF-Transformation

3.2.1 Beschreibung

Der `scan_preprocessor_node` benötigt beim Start die TF-Transformation von `base_link` nach `laser`, um die exakte Montageorientierung des LiDAR-Sensors relativ zum Fahrzeugrahmen zu bestimmen. Diese Transformation codiert den räumlichen Versatz (Translation und Rotation) zwischen Sensor und Fahrzeugmittelpunkt.

Im System war diese Transformation dauerhaft nicht verfügbar. Die entsprechende Fehlermeldung aus dem System-Log:

Listing 3.1: TF-Fehlermeldung aus dem System-Log (28.03.2026)

```
[tf2_echo]: Waiting for transform base_link -> laser:
Invalid frame ID "base_link" passed to canTransform
argument target_frame - frame does not exist
[tf2_echo]: Waiting for transform base_link -> laser:
...
```

3.2.2 Ursache

Die Fahrzeugbeschreibung (URDF) und die davon abgeleiteten TF-Frames werden vom `robot_state_publisher` im Container `mxck2_control` (ROS 2 Humble) publiziert. Der FTG-Stack läuft jedoch im Container `mxck2_development` (ROS 2 Foxy). Da Humble und Foxy verschiedene DDS-Implementierungen verwenden und keine explizite Brücke konfiguriert war, konnten die TF-Nachrichten den Container-Grenzübergang nicht passieren.

Kritische Folge: Der Preprocessor fiel auf einen Fallback-Winkel zurück und konnte die tatsächliche Sensorausrichtung nicht bestimmen. Das bedeutet: Alle nachfolgenden Planungsschritte arbeiteten mit einem *garantiert falschen* Frontbereich.

3.2.3 Beleg aus Messdaten

Das Fehlen des TF-Frames ließ sich direkt verifizieren:

Listing 3.2: Verifikation: kein /tf_static vom FTG-Container aus sichtbar

```
# Aus mxck2_control (Humble) -- liefert Transforms:
ros2 topic echo /tf_static --qos-durability transient_local
# --> transforms: camera_link -> camera_depth_frame, etc.
# --> KEINE base_link -> laser Transform vorhanden!
```

[DATEN / MESSUNG EINFÜGEN]

Screenshot: `ros2 run tf2_tools view_frames` aus dem FTG-Container – zeigt fehlende base_link → laser Kante im TF-Baum

3.3 F2: Falsch kalibriertes Frontfenster

3.3.1 Beschreibung

Der Parameter `front_center_deg` gibt an, unter welchem Winkel im Rohlaserscan die Fahrzeugvorderseite liegt. Der Startwert war 135; nach zwei Kalibrierungsiterationen wurde 0 als korrekte Einstellung ermittelt.

3.3.2 Auswirkung auf den Lenkwinkel

Tabelle 3.2 zeigt den direkten Einfluss des `front_center_deg`-Werts auf den ausgegebenen Lenkwinkel:

Table 3.2: Auswirkung von front_center_deg auf /final_heading_angle

front_center_deg	**/final_heading (rad)**	**Beobachtung**
135 (Startwert)	nicht verfügbar	Keine Lücken gefunden
90	$\approx -1{,}83$ (konstant)	Immer am Limit
0 (Endwert)	+0,18 bis +0,26	Plausibel, variabel

Bei 90 lag der Lenkwinkel konstant bei $-1{,}83$ rad, was exakt dem Grenzwert des Adapters entspricht – ein klares Zeichen, dass der Planerkern in eine Richtung gelenkt wurde, die weit abseits des tatsächlichen Frontbereichs lag.

[FOTO EINFÜGEN]

Plot: /final_heading_angle über Zeit bei verschiedenen front_center_deg-Werten (135°, 90°, 0°) – zeigt die Konvergenz zur stabilen Lösung

Figure 3.1: Einfluss des front_center_deg-Parameters auf den berechneten Zielwinkel

3.3.3 Verbleibende Einschränkung

Auch bei 0 war die Unterscheidung zwischen Hindernispositionen links, mittig und rechts nur begrenzt ausgeprägt (Winkeldifferenz $< 0{,}1\,\mathrm{rad}$). Dies ist ein Hinweis darauf, dass das Frontfenster noch immer nicht perfekt mit der physikalischen Vorderseite des Fahrzeugs ausgerichtet ist – die TF-Transformation (F1) würde diese Feinabstimmung automatisch lösen.

3.4 F3: Zu kleiner Hindernissicherheitsradius

3.4.1 Beschreibung

Im Knoten `obstacle_substitution_node` wird jeder LiDAR-Messpunkt als Kreishindernis mit Radius $r = 0{,}01\,\mathrm{m}$ ($1\,\mathrm{cm}$) modelliert. Das Fahrzeug hat eine Breite von $> 30\,\mathrm{cm}$, weshalb ein Sicherheitsradius von mindestens der halben Fahrzeugbreite ($\approx 0{,}15\,\mathrm{m}$) notwendig wäre.

3.4.2 Konsequenz

Der FTG-Planerkern berücksichtigt den Hindernisradius bei der Lückenbewertung: Eine Lücke gilt als passierbar, wenn ihre Winkelbreite ausreicht, um das Fahrzeug – dargestellt als Kreis mit dem Hindernisradius – hindurchzuführen. Mit $r = 0{,}01\,\mathrm{m}$ werden Lücken als passierbar eingestuft, die in der Realität zu eng für das Fahrzeug sind.

Beispielrechnung: Bei einem Hindernis in $0{,}5\,\mathrm{m}$ Abstand ergibt sich die scheinbar passierbare Lückenbreite aus dem Hindernisradius. Mit $r = 0{,}01\,\mathrm{m}$: freier Winkelbereich $\approx 2\arcsin(0{,}01/0{,}5) \approx 2{,}3$ – viel zu klein, um real einen Unterschied zu machen.

3.5 F4: Fehlende Hindernisgruppierung

3.5.1 Beschreibung

Der Knoten `obstacle_substitution_node` erzeugt für jeden der ≈ 1800 LiDAR-Messstrahlen ein eigenständiges Hindernisobjekt. Eine Wand, die aus 100 benach-

barten Messpunkten besteht, wird als 100 separate Kreishindernisse mit je $r = 0{,}01\,\mathrm{m}$ dargestellt.

3.5.2 Konsequenz

Zwischen diesen 100 dicht beieinanderliegenden Hindernispunkten entstehen 99 sehr enge "Lücken", die der FTG-Algorithmus möglicherweise als Lücken klassifiziert, obwohl die Wand physikalisch nicht durchfahrbar ist. Dies kann zu folgenden Fehlverhaltensmustern führen:

- Falsche Lückenauswahl: Der Algorithmus wählt eine "Lücke" innerhalb einer Wand statt den tatsächlich freien Bereich.
- Zitterndes Lenkverhalten: Die ausgewählte "Lücke" springt zwischen benachbarten Messpunkten, was zu oscillatorischem Lenken führt – genau das im Experiment beobachtete Zickzack-Verhalten.

[FOTO EINFÜGEN]

Foxglove 3D-Ansicht: /obstacles-Topic visualisiert – zeigt die dichte Punktwolke aller Hindernisse (eine Punkt pro LiDAR-Strahl) ohne Clustering

Figure 3.2: Hindernisrepräsentation ohne Clustering – jeder LiDAR-Messpunkt ist ein separates Objekt

3.6 F5: ROS 2-Versionskonflikt zwischen Containern

3.6.1 Beschreibung

Das System nutzt zwei Docker-Container mit verschiedenen ROS 2-Versionen: `mxck2_control` mit Humble und `mxck2_development` mit Foxy. Die Kommunikation zwischen ROS 2-Knoten basiert auf dem DDS-Standard (Data Distribution Service).

3.6.2 Konsequenz

Humble und Foxy verwenden standardmäßig `rmw_fastrtps` als DDS-Implementierung, jedoch mit unterschiedlichen Versionen und Konfigurationsstandards. Ohne explizite Brückenlösung sind zwar `/scan`-Daten über das gemeinsame Docker-Netzwerk übertragbar, jedoch nicht alle Nachrichtentypen – insbesondere nicht TF-`transient_local`-QoS-Nachrichten (`/tf_static`), die für TF-Frames benötigt werden.

[DATEN / MESSUNG EINFÜGEN]

Ausgabe von `ros2 topic list` aus beiden Containern gleichzeitig – zeigt welche Topics container-übergreifend sichtbar sind und welche nicht

3.7 Gesamtbewertung

Table 3.3: Gesamtstatus der Systemkomponenten nach Experimenten

Komponente	Beobachtung	Status
LiDAR-Datenempfang	11 Hz, 360°, stabil	**OK**
obstacle_substitution	Konvertierung funktioniert	**OK**
Scanvorverarbeitung	Nach Kalibrierung plausibel	**Eingeschränkt**
Lückenberechnung	Plausible Winkelwerte	**Eingeschränkt**
Geschwindigkeitsauswahl	Clearance-Stufen funktionieren	**OK**
Ackermann-Ausgabe	Kommandos werden gesendet	**OK**
Kollisionsfreies Fahren	Kollisionen im Test	**Nicht erreicht**
TF-Transformation	Nicht verfügbar	**Fehlt**

3.7.1 Handlungsempfehlungen

Auf Basis der Fehleranalyse werden folgende Maßnahmen priorisiert empfohlen:

1. **[Kritisch] TF-Kette reparieren:** Die `base_link` → `laser`-Transformation muss in dem ROS-Kontext verfügbar sein, in dem der FTG-Stack läuft – entweder durch Zusammenführung der Container oder durch einen `topic_bridge`-Knoten für TF-Nachrichten.

2. **[Hoch] Sicherheitsradius anpassen:** In `obstacle_substitution_node` den Hindernisradius von 0,01 m auf mindestens 0,15 m erhöhen (halbe Fahrzeugbreite).

3. **[Hoch] Clustering hinzufügen:** Einen DBSCAN- oder einfachen Nearest-Neighbor-Clustering-Schritt zwischen `obstacle_substitution` und `follow_the_gap_v0` einführen, um zusammenhängende Hindernisse zu gruppieren.

4. **[Mittel] Container konsolidieren:** Den FTG-Stack in den gleichen Container wie `vehicle_control` migrieren, um alle ROS 2-Topics ohne Versionskonflikte zu teilen.

4 Fazit und Ausblick

4.1 Fazit

Diese Projektarbeit dokumentiert den vollständigen Entwicklungs-, Integrations- und Evaluationsprozess eines Follow-the-Gap-basierten Hindernisvermeidungssystems für das MXCarkit.

Das gesetzte Ziel – ein Fahrzeug, das selbstständig Hindernisse erkennt, ausweicht und bei unausweichlichen Hindernissen anhält – konnte im Rahmen der durchgeführten Experimente *nur teilweise* erreicht werden. Die Einzelkomponenten des Stacks (LiDAR-Empfang, Hinderniskonvertierung, Planerkern, Geschwindigkeitsadapter und Ackermann-Ausgabe) funktionierten im Einzeltest korrekt. Im Zusammenspiel verhinderten jedoch systemische Fehler die kollisionsfreie Navigation.

Zu den wesentlichen Erkenntnissen dieser Arbeit zählen:

- **Kalibrierung ist entscheidend:** Der Parameter `front_center_deg` hatte enormen Einfluss auf das Gesamtverhalten. Eine Abweichung von nur 45 (von 135 auf 90) führte dazu, dass der Algorithmus konstant am Grenzwert operierte und keinerlei situationsabhängige Reaktion zeigte.

- **TF-Infrastruktur ist nicht optional:** Reaktive Planungsalgorithmen hängen fundamental von der korrekten räumlichen Zuordnung der Sensordaten ab. Das Fehlen der `base_link`→`laser`-Transformation verhinderte eine automatische, robuste Kalibrierung und war die tiefste Ursache des Systemversagens.

- **Hindernismodellierung beeinflusst Lückenbewertung:** Ein Sicherheitsradius von 0,01 m ist für ein Fahrzeug der Breite > 30 cm unzureichend. Die Wahl des Hindernisradius direkt beeinflusst, welche Lücken als passierbar gelten.

- **Modularität erleichtert Debugging:** Die konsequent modulare Paketstruktur mit dedizierten Debug-Topics (`/visualize_obstacles`, `/autonomous/ftg/planner_status`) ermöglichte eine systematische Eingrenzung der Fehlerquellen – ohne diese Transparenz wäre das Debugging erheblich schwieriger gewesen.

- **Container-Grenzen sind ROS-Grenzen:** Das DDS-basierte Kommunikationsmodell von ROS 2 funktioniert gut innerhalb eines Netzwerks, jedoch entstehen durch unterschiedliche ROS 2-Versionen und QoS-Inkompatibilitäten unsichtbare Kommunikationsgrenzen.

4.2 Ausblick

Mit den in Kapitel 3 beschriebenen Korrekturen ist zu erwarten, dass das System die ursprüngliche Zielsetzung – kollisionsfreies Fahren mit Nothalt – vollständig erfüllen kann. Über die unmittelbaren Korrekturen hinaus bieten sich folgende Erweiterungen an:

1. **Clustering-Integration:** Ein DBSCAN-basierter Clusteringschritt zwischen `obstacle_subst` und dem Planerkern würde die Hindernisrepräsentation erheblich verbessern und das beobachtete Zickzack-Lenkverhalten reduzieren.

2. **Systematische Parametrierung via Rosbag:** Die aufgezeichneten Rosbags können für offline Parametertests genutzt werden, ohne das reale Fahrzeug jedes Mal neu starten zu müssen.

3. **Foxglove-Dashboard:** Eine Foxglove-Konfiguration mit 3D-Panel (Hindernisvisualisierung, Heading-Pfeil), Plot-Panel (`/final_heading_angle`, `/gap_found`) und Raw-Messages-Panel würde zukünftige Kalibrierung und Diagnose erheblich erleichtern.

4. **Streckentest:** Erprobung des Systems auf einer definierten Teststrecke mit bekannten Hindernishindernissen in verschiedenen Konfigurationen (Engstellen, L-Kurven, dynamische Hindernisse).

5. **Geschwindigkeitsskalierung:** Nach erfolgreicher Kollisionsfreiheit kann die `cruise_speed` schrittweise erhöht und die Geschwindigkeitsstufen feinjustiert werden.

Bibliography

[1] S. Sezer und M. Gokasan, *Novel obstacle avoidance algorithm for short range ultrasonic sensors*, Journal of the Franklin Institute, Bd. 349, Nr. 6, S. 1939–1958, 2012.

[2] F1TENTH-Community, *F1TENTH Autonomous Racing – Follow the Gap Algorithm*, Online-Dokumentation, 2023. Verfügbar: `https://f1tenth.readthedocs.io`

[3] Open Robotics, *ROS 2 Foxy Fitzroy – Offizielle Dokumentation*, 2020. Verfügbar: `https://docs.ros.org/en/foxy/`

[4] Slamtec, *RPLIDAR A1/A2 – Low Cost 360° Laser Range Scanner, Technisches Datenblatt*, 2023.

[5] R. Rajamani, *Vehicle Dynamics and Control*, 2. Aufl., Springer, New York, 2012.

[6] W. Meisenheimer, *mxck2_ws – MXCK2 Robot Operating System Workspace*, GitHub-Repository, 2024. Verfügbar: `https://github.com/william-mx/mxck2_ws`

Eidesstattliche Erklärung

Wir erklären hiermit, dass wir die vorliegende Projektarbeit selbstständig und ohne unzulässige Hilfe Dritter verfasst haben. Alle verwendeten Quellen und Hilfsmittel sind vollständig angegeben. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Friedberg, April 6, 2026

Aly Elbermawy	Omar Elalami	Zakaria El-aboudi